

# SMART OFFICE 2.0

## Backlog & Gestion de projet Agile

---

### Équipe projet

Sam Flaujat-Ellul | Alexander Kedevil

### Formation

Bachelor 3 — Spécialité Infrastructure & Réseaux / Cybersécurité

### Établissement

Ynov Campus Montpellier — UF INFRA B3

Juin 2026

## Introduction

Ce document présente l'organisation agile du projet Smart Office 2.0 : structure du backlog, découpage en sprints, suivi des user stories et historique des contributions GitHub. La méthodologie adoptée est un hybride Scrum/Kanban, géré via Trello et versionné sur GitHub.

*L'agilité est un critère d'évaluation explicite de l'UF INFRA B3. L'hybride Scrum/Kanban est choisi car il combine la cadence des sprints (Scrum) avec la flexibilité des tickets en flux continu (Kanban), adaptée à une équipe de 2 personnes sans Scrum Master dédié. Trello remplace Jira pour sa gratuité et sa prise en main immédiate.*

## 1. Méthodologie et outils

- Framework : Scrum / Kanban hybride
- Outil de suivi : Trello — board b3-infra
- Versionnage : GitHub — dépôt ynov-b3-infra
- Communication : Discord / Microsoft Teams

### 1.1 Structure du board Trello

| Colonne  | Usage                                                                                            |
|----------|--------------------------------------------------------------------------------------------------|
| Backlog  | Évolutions hors scope PoC (VPN Azure, HA, SIEM avancé) — non engagées dans la livraison actuelle |
| En cours | Tâches actives pendant la période de sprint courante                                             |
| Fait     | Livrables mergés : réseau, cloud, IAM, monitoring, docs                                          |

*La colonne Backlog conserve les fonctionnalités identifiées mais hors scope PoC (VPN Azure, HA, SIEM Wazuh). Cela documente la vision cible tout en maintenant le focus sur les livrables notés. Le principe MoSCoW (Must/Should/Could/Won't) est implicitement appliqué.*

## 2. Planification des sprints

Chaque sprint a une durée de 4 à 6 semaines, calée sur les jalons pédagogiques du cursus B3 :

| Sprint | Période        | Objectif                    | Livrables                                                                      |
|--------|----------------|-----------------------------|--------------------------------------------------------------------------------|
| S1     | Fév.–Mars 2026 | Architecture réseau on-prem | pfSense, 6 VLANs, VMware vmnet2, plan IP, captures firewall, schéma réseau     |
| S2     | Mars–Avr. 2026 | PoC cloud + IAM             | room-booking Flask, PostgreSQL/Redis, Entra ID (code), docker-compose          |
| S3     | Avr.–Mai 2026  | Azure + CI/CD               | ACR, ACI, GitHub Actions, corrections déploiement (limite mémoire, ACR mirror) |
| S4     | Mai–Juin 2026  | Supervision + documentation | Grafana/Loki, syslog pfSense, DAT, BIA, PCA/PRA, Merise                        |
| S5     | Juin 2026      | Clôture projet              | Trello final, export Moodle, soutenance                                        |

*La séquence des sprints suit une logique de dépendances techniques : le réseau on-prem (S1) est le prérequis à l'application (S2), elle-même prérequis au déploiement cloud (S3). La supervision (S4) vient en dernier car elle nécessite des sources de logs stables. Cette priorisation réduit les risques de blocage en cascade.*

### 3. User stories

Les user stories sont formulées du point de vue de l'équipe projet jouant le rôle d'administrateur système. Chaque story correspond à un livrable évaluable selon la grille UF INFRA B3 :

| ID    | Story                            | Critère d'acceptation                                                        |
|-------|----------------------------------|------------------------------------------------------------------------------|
| US-01 | Segmenter le réseau en VLANs     | 6 VLANs opérationnels, règles pfSense configurées, captures d'écran fournies |
| US-02 | Réserver une salle via API       | CRUD rooms/bookings fonctionnel, cache Redis opérationnel                    |
| US-03 | Déployer l'application sur Azure | ACI healthy, pipeline GitHub Actions vert                                    |
| US-04 | Authentifier via Entra ID        | JWT + MSAL (démon AUTH_DISABLED si erreur 401 tenant Ynov)                   |
| US-05 | Superviser les anomalies         | Dashboard Grafana opérationnel, logs pfSense + API collectés                 |
| US-06 | Documenter l'architecture        | DAT, PCA/PRA, BIA, Merise, TCO produits et versionnés sur GitHub             |

*Le critère d'acceptation est volontairement technique et mesurable (« ACI healthy », « pipeline GitHub Actions vert ») pour éviter toute ambiguïté lors de la validation. US-04 inclut une alternative (AUTH\_DISABLED) documentant explicitement la contrainte du tenant Ynov, preuve de rigueur et de transparence.*

### 4. Historique des Pull Requests GitHub

Les pull requests constituent la trace versionnée de chaque livrable technique. Chaque PR correspond à une ou plusieurs user stories :

| PR        | Thème et apport                                                                                    |
|-----------|----------------------------------------------------------------------------------------------------|
| PR #12    | Docs prep, README, skeleton docs/ — mise en place de la structure de documentation                 |
| PR #13    | room-booking Flask + PostgreSQL + Redis — cœur applicatif du service de réservation                |
| PR #14    | Entra ID auth, Zero Trust docs — intégration IAM et documentation sécurité                         |
| PR #15    | Azure ACI deploy + workflow GitHub Actions — automatisation du pipeline CI/CD                      |
| PR #16–18 | Corrections workflow, ACI provider, ACR sidecar images — stabilisation de la chaîne de déploiement |

*L'usage systématique des PRs (plutôt que des commits directs sur main) impose une revue avant merge et documente l'intention de chaque modification. Les PRs #16 à #18 témoignent d'une démarche itérative et réaliste : les bugs de déploiement ACI ont été corrigés et versionnés, démontrant la capacité de l'équipe à déboguer un pipeline CI/CD en production.*

*Dépôt GitHub : <https://github.com/AlexKedevil/ynov-b3-infra>*